DATA PRODUCT CASE STUDY

# PRIME: one governed data layer for a multi-tenant product

How StellarAlgo got Customer Success, Product and clients working from the same numbers, and the same process run on City of Calgary transit data.

**Cody Chandler**
Data and Business Analyst interview · The City of Calgary

# The order I worked in, and where each question lands

1

### The problem

Q1 · why it mattered, who it affected

2

### What people needed

Q1 · the users

3

### Strategy

Q3 · the approach

4

### Plan

Q3 · sourcing and sequence

5

### Build

Q2 · what it does · Q3 · tools

6

### Review and iterate

Q2 · the value

### Then, live on Calgary data

The same process on City open data: dashboard and AI agent

### Challenges

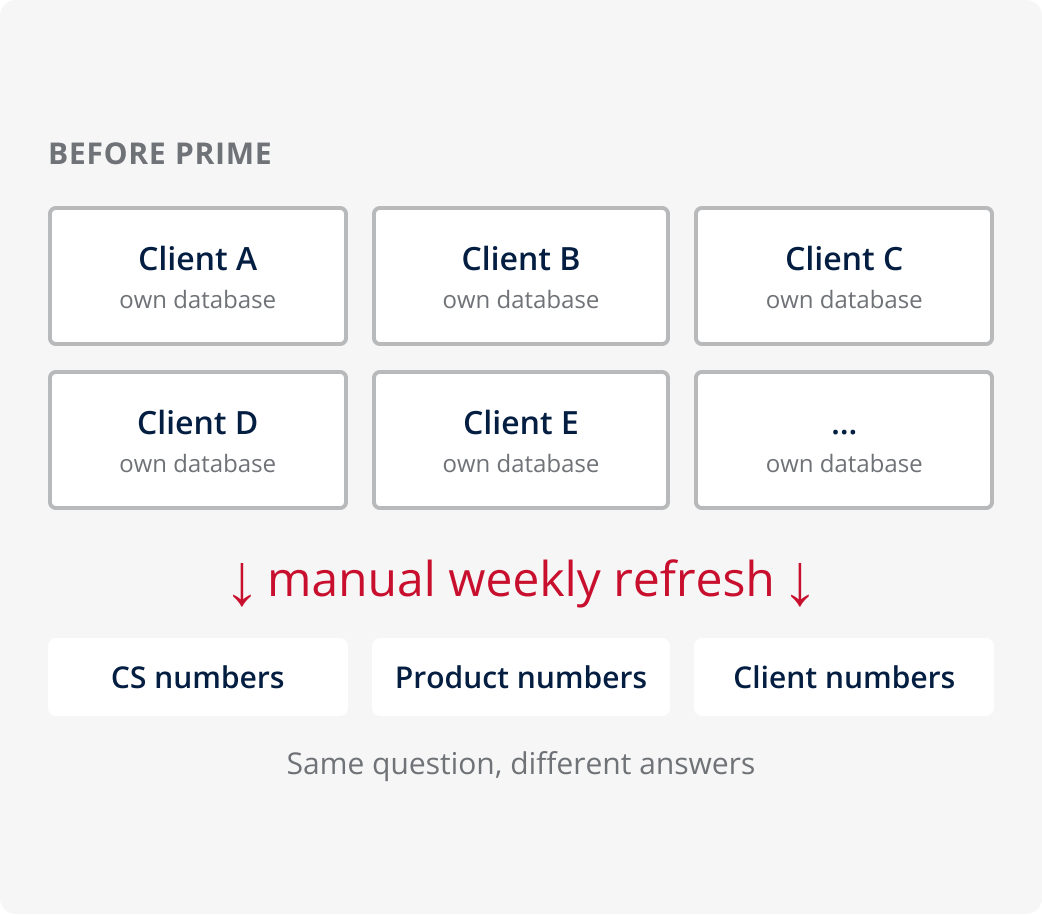
Q4 · technical, analytical and people

### Proudest feature and what's next

Q5 and Q6

# Multi-tenancy made accurate reporting hard

Every client's data lived in its own database. Any question across clients meant querying each one separately.




### Stale

A manual weekly refresh meant data could be up to 7 days old.

### Fragmented

Cross-property questions meant querying every client system separately, through engineering.

### Unowned

Ownership was split across teams, with no agreed service levels when something broke.

### Why it mattered

Decisions and client conversations ran on numbers nobody could fully stand behind, and AI assistants would only repeat them.

# Each group's need became a requirement

| Who | What they needed | Requirement it became |
|---|---|---|
| **Customer Success** | Answers for clients without waiting on engineering | **Self-serve for routine questions** |
| **Product** | A view across every client, not one at a time | **One definition across all clients** |
| **Clients** | Dashboards and data shares they can rely on | **Fresh, reconciled data on a known schedule** |
| **Analysts** | Clean, documented models to build on | **Tested and documented, every run** |
| **AI assistants** | Tables with no ambiguity about what a row means | **One declared grain per table** |
| **Everyone** | Someone accountable when it breaks | **Named owners, targets and incident tiers** |

Every design decision that follows traces back to one of these rows.

# Fix the layer, not the reports

Four decisions shaped everything after them.

## 1 Consolidate once

One governed Snowflake layer for every client, instead of patching each client database and each report.

## 2 Agree the contract before building

Refresh and accuracy targets, incident tiers and owners, agreed with the teams who rely on them.

## 3 Model by grain

Every consumption table answers one shape of question, so neither people nor AI double count.

## 4 Prove AI internally first

An internal Customer Success bot before any client-facing assistant.

# Sequenced so every step was usable on its own

**1**

## Agree

The contract and the owners

**2**

## Land and protect

All sources into bronze, personal data masked, roles defined

**3**

## Define once

Silver: one dimensional model across clients

**4**

## Serve

Gold for the CS platform, data sharing and dashboards

**5**

## Watch

Freshness every 30 minutes, error digests, cost per model

**6**

## Add AI

Internal CS bot, then Stellar Insights discovery

**SOURCES**

Ticketing and accounts · app configuration and logs · automations · segments · client and league reference files

# PRIME: one governed layer every consumer reads from

SOURCES
**Ticketing, accounts, apps**
Logs, automations, segments, reference files

→

BRONZE
**Raw, as landed**
Personal data masked on arrival

→

SILVER
**One definition**
Dimensional model across all clients

→

GOLD
**Consumption tables**
Rebuilt every run, tested first

→

CONSUMERS
**CS · clients · BI · AI**
Role-based access by consumer

### Data contract

Refresh and accuracy targets per domain, three incident tiers, a named owner for each.

### Governed access

Personal data masked at the raw layer; analysts, client dashboards and AI each get their own role.

### Monitoring

Daily Airflow and dbt refresh; freshness checked every 30 minutes with AI-written error digests.

**MY ROLE** Director of Product Innovation & Analytics. I owned PRIME's product management from planning through general availability, working with Data Engineering and Customer Success through a small product pod.

# How people use it: ask a question, get the right grain

A CS MANAGER ASKS

**“How did ticket revenue trend this month?”**

The agent picks the semantic model whose **grain** fits the question and answers from it. Answering a campaign question from a daily table would double-count clicks.

THE SAME GOLD TABLES FEED

Client success platform · Client data sharing · BI dashboards · AI assistants

| Semantic model | One row per… |
|---|---|
| Team trends | team per day |
| Campaigns | email campaign |
| Segments | audience segment |
| Users | user per day |
| Automations | automation |

# What changed, and what we kept improving

~~7 days~~

**< 12 hrs**

Data freshness

**100+**

Tested dbt models, replacing a database per client

**30 min**

To flag a missed refresh

**Self-serve**

Routine Customer Success questions

### Trust before scale

The CS bot shipped first, so accuracy was proven on our own people before clients.

### Grain as a rule

Every consumption table declares its grain, because the wrong table silently double counts.

### Watched, not hoped

Missed refreshes surface in 30 minutes, with cost tracked per model.

LIVE ON CITY DATA

# PRIME's data belongs to StellarAlgo's clients. So I ran the same process on Calgary's open data.

Same steps. Real City data. Refreshed daily, and live in this browser.

# Calgary Transit, the same six steps

## 1 The problem

Ridership, service hours and 311 are published separately, with shifting codes and no shared view of how current each is.

## 2 What people needed

Service planners, 311 leads, fare policy and leadership: one trusted trips number, comparable trends, known freshness.

## 3 Strategy

Fix the data once. Contract before charts. One grain per table. Only gold leaves the warehouse.

## 4 Plan

Three datasets, five gold tables, one per question. Land, map, test, publish, then dashboard and agent.

## 5 Build

Airflow (31 tasks), dbt (10 models, 27 tests), DuckDB, a daily GitHub Actions run, and this site.

## 6 Review and iterate

The first cloud run failed and was fixed. A 43% jump in complaints turned out to be a new 311 channel.

# The build, end to end

SOURCE
**Calgary open data**
Ridership, service hours, 271K transit 311 requests

→

BRONZE
**Airflow**
Raw JSON, incremental 311 loads, load log

→

SILVER
**dbt**
One definition across reorganizations

→

GOLD
**dbt**
5 tables, one declared grain each

→

CONSUMERS
**Dashboard · AI agent**
Read gold only

**Sourcing:** the City's Socrata API; 311 loads incrementally with a 7-day lookback.

**Preparation:** reference tables map 4 department codes and 67 request types.

**Quality:** 27 tests every run; a failure stops publishing.

**Portability:** DuckDB here; the same dbt code runs on Snowflake.

# The hard parts were definitions and people, not code

AT STELLARALGO

### Technical: one definition across clients

Every client system had its own shape. Silver became one dimensional model all properties map into.

### Analytical: grain

The wrong table silently double counts. Each model declares one grain, and the agent chooses by it.

### People: nobody owned freshness

Split ownership, no service levels. The contract named owners, targets and incident tiers.

IN THE CITY'S DATA, THIS WEEK

### One department, four codes

Calgary Transit appears in 311 as TRAN, OS, OSC and Tranc. A reference table unifies them.

### A misleading trend

Complaints per trip looked up 43%. Lost Property moved into 311 in 2021; like for like, it's 12%.

### Totals that don't add up

Fare products never sum to the published total. The gap is an explicit, tested row.

# AI answers that match the dashboard

An AI assistant is only as good as the layer under it. One documented grain per table and a tested contract are what make plain-English answers trustworthy.

### Reads the docs, not the raw data

Its instructions come from the dbt documentation: grains, column meanings, and rules like “never add trips to boardings”.

### Shows the SQL

Every answer displays the exact query and rows behind it, so anyone can check the number.

### Can't touch what it shouldn't

Read-only queries on gold tables only, with file and network access switched off.

YOU ASK

**“How has ridership recovered since the pandemic?”**

```
-- the kind of query it writes, shown with every answer
select year, sum(trips) as trips
from transit_monthly_kpis
where has_ridership_data
group by year
having count(*) = 12   -- complete years only
order by year;
```

# What I'd build next

## PRIME

### Stellar Insights for clients

Take plain-English answers from the internal CS bot to clients, on the same semantic layer.

### Contract health as a product metric

Report freshness and incident history to the teams that own each domain.

## CALGARY TRANSIT

### On-time performance

Land the City's real-time vehicle positions to measure reliability by route.

### An equity lens

Join requests and service to the Calgary Equity Index by community.

### Owner sign-off

Agree freshness targets and category mappings with the people who run the service.

The same work at the City: find where numbers disagree, agree one definition with the people who own it, and make it self-serve.

# Thank you. Questions?

Everything shown today is public: the dashboard, the agent, the pipeline code and the dbt documentation.

| | |
|---|---|
| Code | **github.com/frontier-platforms/data-product-use-case** |
| dbt docs | **frontier-platforms.github.io/data-product-use-case** |